

National Autonomous University of Mexico

Faculty of Engineering

Compilers

Group 5

Lexical Analyzer (Lexer)

Technical Report

Team 07

Account number	Role
321132422	Presenter
321237125	Technical Writer
321203018	Developer
321135344	Model Designer
321193692	Project Manager

Repository: <https://github.com/HansZuniga/Team-07>

Package: `unam.fi.compilers.g5.07`

October 6, 2026

Contents

1	Introduction	2
2	Problem Statement and Justification	2
2.1	Problem Statement	2
2.2	Justification	2
3	Objectives	3
3.1	General Objective	3
3.2	Specific Objectives	3
4	Theoretical Background	3
4.1	The Lexical Analyzer within the Compiler	3
4.2	Token, Lexeme and Pattern	3
4.3	Regular Expressions and Finite Automata	4
4.4	Longest Match and Priority	4
4.5	Lexical Errors and Panic Mode	4
4.6	Context-Free Grammars and Left Factoring	4
5	Development	4
5.1	Work Methodology	4
5.2	Token Specification	5
5.3	NFA/DFA Model	5
5.3.1	Identifiers	5
5.3.2	Numeric Constants	6
5.3.3	Relation to the Program	7
5.4	Context-Free Grammar and Left Factoring	7
5.5	Implementation	8
5.5.1	Package Architecture	8
5.5.2	Compiled Patterns	9
5.5.3	Scanning Algorithm	9
5.5.4	Lexical Error Handling	10
5.5.5	Input and Output	10
6	Results	11
6.1	Professor's Reading Example	11
6.2	Reserved Words and Two-Character Operators	12
6.3	Lexical Errors	12
6.4	Automated Tests	13
7	Analysis of Results	15
7.1	Requirements Compliance	15
7.2	Relation between Theory and Implementation	15
7.3	Findings during Testing	15
7.4	Limitations	15
8	Conclusions	16
	References	17

1 Introduction

A compiler translates a program written in a source language into an equivalent program in another language. This translation is organized in phases, and the first of them is **lexical analysis**: the source program arrives as a plain sequence of characters, and the lexical analyzer (*lexer* or *scanner*) groups it into meaningful units called *tokens*, which are then consumed by the syntax analyzer [1].

This report documents the lexical analyzer developed by Team 07 for the Compilers course. The program is written in Python 3, does not use the FLEX tool, and is based on the concepts studied in class: regular expressions, finite automata and the longest-match policy. It receives a string or a file, recognizes six token categories (**KEYWORD**, **IDENTIFIER**, **CONSTANT**, **LITERAL**, **OPERATOR** and **PUNCTUATION**), reports the line and column of each token, reports lexical errors without stopping the analysis, and displays the total number of tokens found.

The document first presents the problem, the objectives and the theoretical background; it then describes the formal model built by the team (token specification, automata, and a context-free grammar with left factoring) and its implementation; finally, it shows the execution and test results, their analysis, and the conclusions.

2 Problem Statement and Justification

2.1 Problem Statement

The project statement [5] asks for a lexical analyzer that:

- reads an input string or file;
- presents basic tokens: at least the keyword **print**, as well as identifiers;
- displays the sequence of recognized token categories and the **total number of tokens**;
- processes the reading examples given in the statement (`printf("This is an example");` and `int a = 10;`), written here with straight quotes (see Section 7.4);
- does not use FLEX, either by applying only the theoretical concepts seen in class or by using libraries of the programming language;
- includes, theoretically, a basic context-free grammar (CFG) that represents one of the reading examples, enhanced through *left factoring*;
- is delivered packaged as `unam.fi.compilers.g5.07` in a public repository.

The core problem is therefore to turn unstructured text into a correctly classified sequence of tokens, resolving the cases in which the same fragment could belong to several categories (for example, `int` versus `intx`, or `>` versus `>=`), and handling in a controlled way the characters that do not belong to the language.

2.2 Justification

The lexical analyzer is the foundation of the whole compiler: if tokens are misclassified, no later phase can work correctly. Moreover, separating lexical analysis from syntax analysis simplifies the design of the compiler, improves its efficiency, and isolates the particularities of the input alphabet [1].

From an academic point of view, the project makes it possible to apply in a concrete way the theoretical chain *regular expression* $\rightarrow$ *NFA* $\rightarrow$ *DFA*, as well as the concepts of context-free grammars and left factoring.

Python was chosen because it allows the team to focus on the lexical analysis logic and because its `re` module [3] implements regular expressions in a standard way, so that each pattern in the program corresponds directly to a regular language in the theoretical model. A hand-written scanner was preferred over a lexer generator in order to control precisely the line and column of each lexeme and the error recovery policy.

3 Objectives

3.1 General Objective

To design and implement in Python, without using FLEX, a lexical analyzer that receives a string or a file, identifies its lexemes, classifies them into token categories, reports lexical errors, and displays the total number of valid tokens, supporting its behavior on the theoretical model seen in class.

3.2 Specific Objectives

1. Define the token catalog of the language with its regular expressions, examples and priority rules.
2. Model the representative tokens `IDENTIFIER` and `CONSTANT` with an NFA and a DFA, applying ε -closure and the subset construction.
3. Propose a CFG for the example `int a = 10;` and apply left factoring to it.
4. Implement the scanner using the longest-match policy and panic-mode error recovery.
5. Validate the program with an automated test suite that includes the professor's examples.

4 Theoretical Background

4.1 The Lexical Analyzer within the Compiler

Figure 1 shows where the lexical analyzer is located. It reads the source program character by character, groups the characters into lexemes, and produces for each one a token that is passed to the syntax analyzer. Along the way, it discards what is not relevant to the syntax, such as spaces, tabs and newlines, and keeps track of the position so that errors can be reported [1].

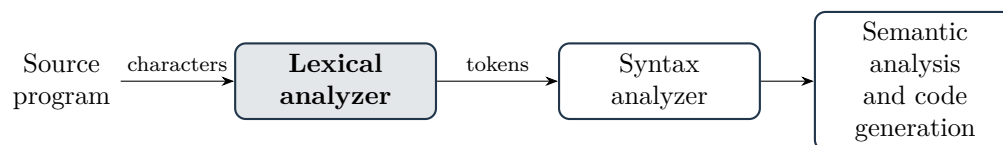


Figure 1: Position of the lexical analyzer among the phases of the compiler.

4.2 Token, Lexeme and Pattern

- **Lexeme:** a concrete sequence of characters in the source program, for example `counter` or `10`.
- **Token:** the category a lexeme belongs to, for example `IDENTIFIER` or `CONSTANT`. Formally, it is a pair made of the token name and an optional attribute value, such as the lexeme itself and its position [1].
- **Pattern:** a description of the form the lexemes of a token can take; in this project, a regular expression.

4.3 Regular Expressions and Finite Automata

Token patterns describe **regular languages**, which can be expressed with regular expressions and recognized by finite automata [2]. A **nondeterministic finite automaton (NFA)** may have several transitions on the same symbol and ε -transitions that consume no input; a **deterministic finite automaton (DFA)** has exactly one transition per symbol from each state. Every NFA can be converted into an equivalent DFA through:

- **ε -closure**, $E(q)$: the set of states reachable from q using only ε -transitions.
- **Subset construction**: each DFA state is a set of NFA states; from a set A and a symbol a the DFA moves to $E(\text{move}(A, a))$. A DFA state is accepting if it contains at least one accepting NFA state.

4.4 Longest Match and Priority

When several patterns could recognize a prefix of the input, lexical analyzers apply two rules [1]: the **longest** possible lexeme is chosen (*maximal munch*) and, if two patterns match the same length, the one with **higher priority** wins. Thus, `intx` is one complete identifier and not `int` followed by `x`, and `>=` is a single operator.

4.5 Lexical Errors and Panic Mode

A lexical error occurs when no pattern matches the prefix of the input. The simplest recovery strategy is **panic mode**: the error is recorded, characters are discarded until scanning can continue, and the analysis goes on, so that one error does not prevent checking the rest of the program [1].

4.6 Context-Free Grammars and Left Factoring

A context-free grammar is a tuple $G = (N, T, P, S)$ with nonterminals N , terminals T , productions P of the form $A \rightarrow \alpha$ with $A \in N$, and a start symbol S . When two alternatives of the same nonterminal share a prefix, $A \rightarrow \alpha\beta_1 \mid \alpha\beta_2$, a top-down parser cannot decide which one to use by looking only at the next token. **Left factoring** extracts the common prefix [1]:

$$A \rightarrow \alpha A', \quad A' \rightarrow \beta_1 \mid \beta_2.$$

The resulting grammar generates the same language, but allows the decision to be postponed until the common prefix has been read.

5 Development

5.1 Work Methodology

The team worked with a Kanban board on GitHub (columns *Backlog*, *Ready*, *In progress*, *Review* and *Done*) and integrated changes through pull requests. The roles were: Manager (planning, repository, scope and integration), Model Designer (token specification, automata and CFG), Developer (implementation), Developer and Model Designer together (tests), Technical Writer (this report) and Presenter (defense and demonstration). Design decisions were recorded in `docs/decisions.md` and the formal model in the `docs/` folder of the repository [6].

5.2 Token Specification

Table 1 summarizes the team's token specification as implemented in the final program. The only difference from the written specification is in **LITERAL**: the specification also allowed curly quotes, but the final implementation accepts only straight quotes (see Section 7.4). The lexer is case-sensitive, and the identifier alphabet is limited to ASCII letters, digits and the underscore. Reserved words are read from `resources/keywords.txt`.

Category	Regular expression	Valid / invalid examples
KEYWORD	<code>print int return if else</code>	<code>int, print</code> / <code>intx, printf</code> (identifiers)
IDENTIFIER	<code>[A-Za-z_][A-Za-z0-9_]*</code>	<code>main, _x, if2</code> / <code>2a</code>
CONSTANT	<code>[0-9]+(?:\.[0-9]+)?</code>	<code>10, 0, 3.14</code> / <code>3., 123abc, 1.2.3</code>
LITERAL	<code>"[^\"\\r\\n]*"</code>	<code>"Hello", ""</code> / unterminated string
OPERATOR	<code>== != <= >= [+=*/<>-]</code>	<code>=, >=, + / &, !</code>
PUNCTUATION	<code>[() { } ; ,]</code>	<code>(, ;, }</code> / <code>[,]</code>

Table 1: Token catalog and regular expressions.

Priority rules and conflicts:

- An identifier is read completely before it is reclassified: if the lexeme belongs to the set of reserved words it is a **KEYWORD**; otherwise it is an **IDENTIFIER**. Therefore `if` is a **KEYWORD** and `if2` is an **IDENTIFIER**.
- Two-character operators (`=`, `!=`, `<=`, `>=`) are tried before one-character operators.
- Signs are independent operators: `-10` produces an **OPERATOR** and a **CONSTANT**.
- A string is recognized as a single unit: spaces and operators inside it do not produce additional tokens.
- Spaces, tabs and newlines are not tokens, but they update the position. Lines and columns start at 1.

5.3 NFA/DFA Model

The two most representative patterns, **IDENTIFIER** and **CONSTANT**, were modeled. In the diagrams, the *start* arrow marks the initial state and the double circle marks an accepting state. In the tables, $M = \emptyset$ is the dead state, which absorbs any character.

5.3.1 Identifiers

Pattern `[A-Za-z_][A-Za-z0-9_]*`. Let L be an ASCII letter or underscore and D a digit. The NFA has $Q = \{q_0, q_1, q_2\}$, initial state q_0 and accepting states $F = \{q_2\}$, with $\delta(q_0, \varepsilon) = \{q_1\}$, $\delta(q_1, L) = \{q_2\}$ and $\delta(q_2, L) = \delta(q_2, D) = \{q_2\}$ (Figure 2).

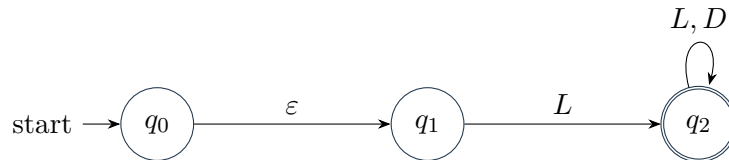


Figure 2: NFA for **IDENTIFIER**.

The closures are $E(q_0) = \{q_0, q_1\}$, $E(q_1) = \{q_1\}$ and $E(q_2) = \{q_2\}$. The initial DFA state is $A = E(\{q_0\}) = \{q_0, q_1\}$; on reading L , $E(\text{move}(A, L)) = \{q_2\} = B$, and from B , reading L or D stays in B . B is accepting because it contains q_2 . The result is shown in Table 2 and Figure 3.

State	L	D	Other
$\rightarrow A = \{q_0, q_1\}$	B	M	M
$*B = \{q_2\}$	B	B	M
$M = \emptyset$	M	M	M

Table 2: Transition table of the DFA for IDENTIFIER.

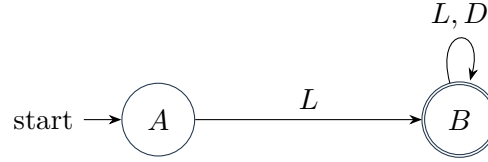


Figure 3: DFA for IDENTIFIER (the dead state M is omitted).

Check. main: $A \rightarrow B \rightarrow B \rightarrow B \rightarrow B$, accepted; $_x$ and $\text{if}2$ are also accepted. **2a:** $A \rightarrow M \rightarrow M$, rejected. The empty string stays in A , which is not accepting.

5.3.2 Numeric Constants

Pattern $[0-9]^+(\backslash.[0-9]^+)?$. Let d be a digit. The NFA has $Q = \{q_0, \dots, q_5\}$, initial state q_0 and accepting states $F = \{q_2, q_5\}$ (Figure 4):

$$\begin{aligned} \delta(q_0, \varepsilon) &= \{q_1\}, & \delta(q_1, d) &= \{q_2\}, & \delta(q_2, d) &= \{q_2\}, \\ \delta(q_2, \varepsilon) &= \{q_3\}, & \delta(q_3, \cdot) &= \{q_4\}, & \delta(q_4, d) &= \{q_5\}, & \delta(q_5, d) &= \{q_5\}. \end{aligned}$$

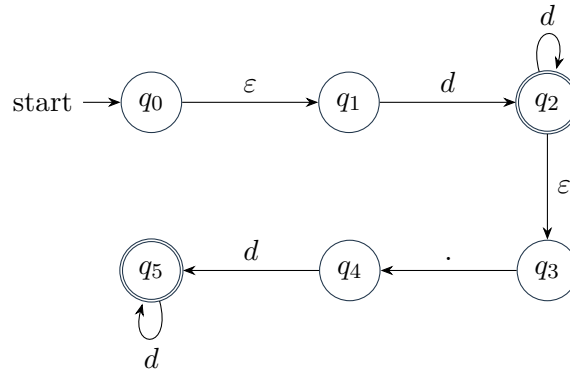


Figure 4: NFA for CONSTANT.

The non-trivial closures are $E(q_0) = \{q_0, q_1\}$ and $E(q_2) = \{q_2, q_3\}$. By the subset construction: $S = \{q_0, q_1\}$ is the initial state; $E(\text{move}(S, d)) = \{q_2, q_3\} = N$; from N , a digit stays in N and a

dot leads to $P = \{q_4\}$; from P , a digit leads to $F_d = \{q_5\}$, which is kept with further digits. N and F_d are accepting (Table 3 and Figure 5).

State	d	Dot	Other
$\rightarrow S = \{q_0, q_1\}$	N	M	M
$*N = \{q_2, q_3\}$	N	P	M
$P = \{q_4\}$	F_d	M	M
$*F_d = \{q_5\}$	F_d	M	M
$M = \emptyset$	M	M	M

Table 3: Transition table of the DFA for **CONSTANT**.

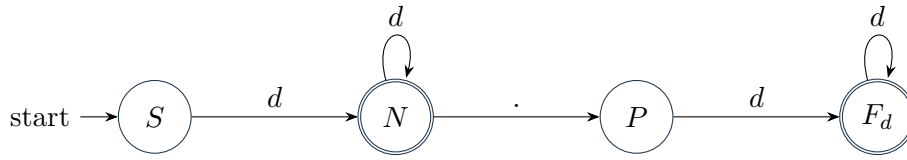


Figure 5: DFA for **CONSTANT** (the dead state M is omitted).

Check. 10: $S \rightarrow N \rightarrow N$, accepted. 3.14: $S \rightarrow N \rightarrow P \rightarrow F_d \rightarrow F_d$, accepted. 3. ends in P , which is not accepting; 123abc and 1.2.3 reach M and are rejected as constants.

5.3.3 Relation to the Program

The regular expressions `_IDENTIFIER_RE` and `_CONSTANT_RE` in the program describe exactly the languages recognized by these DFAs. Python's `re` module runs each pattern with its own matching engine, so the program does not build the automata at run time; the model documents the theoretical equivalence $Regex \rightarrow NFA \rightarrow DFA$ seen in class. The automata decide whether a complete candidate belongs to the language; the scanner, in addition, delimits that candidate, applies the longest match, and reports errors.

5.4 Context-Free Grammar and Left Factoring

The professor's example `int a = 10;` was chosen, together with the declaration without initialization `int a;`, so that there are two alternatives with a common prefix. The grammar works on token categories: *id* stands for an identifier and *num* for a numeric constant, both of them terminals.

Original grammar. $G_0 = (N_0, T, P_0, S)$ with $N_0 = \{S\}$, $T = \{\text{int}, id, num, =, ;\}$ and

$$S \rightarrow \text{int } id = num ; \mid \text{int } id ;$$

It is context-free because every production has a single nonterminal on its left-hand side.

Left factoring. Both alternatives share the prefix `int id`. This prefix is extracted and the non-terminal R is introduced for the rest:

$$\begin{aligned} S &\rightarrow \text{int } id R \\ R &\rightarrow = num ; \mid ; \end{aligned}$$

The new grammar is $G_1 = (N_1, T, P_1, S)$ with $N_1 = \{S, R\}$. $R \rightarrow ;$ is not an empty production: it generates the semicolon of the declaration without initialization.

Derivations.

$$S \Rightarrow \text{int } id \ R \Rightarrow \text{int } id = num \ ; \qquad S \Rightarrow \text{int } id \ R \Rightarrow \text{int } id \ ;$$

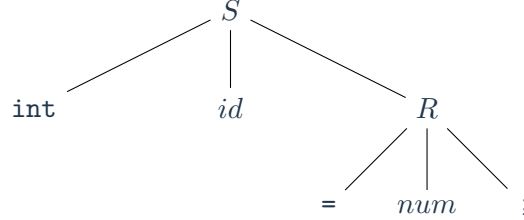


Figure 6: Parse tree of `int a = 10;` with the factored grammar.

The leaves of the tree (Figure 6), from left to right, are `int`, `id`, `=`, `num`, `;`, which match exactly the token sequence produced by the lexer for `int a = 10;`: `KEYWORD IDENTIFIER OPERATOR CONSTANT PUNCTUATION` (Section 6).

Language preservation. $L(G_0) = \{\text{int } id = num \ ;, \text{int } id \ ;\}$. In G_1 , S generates the common prefix and R generates exactly the two original suffixes, so $L(G_0) = L(G_1)$. The transformation reorganizes the productions without adding or removing structures, and allows a top-down parser to choose the production for R by looking at a single token (`=` or `;`). This grammar covers simple `int` declarations; it is not intended to be a complete grammar for C, and the lexer does not include a parser.

5.5 Implementation

5.5.1 Package Architecture

The repository uses the reverse-domain naming convention, so the requested package `unam.fi.compilers.g5.07` is located under the folder `mx/unam/fi/compilers/g5/07`. The package separates code, resources, examples and tests:

```

mx/unam/fi/compilers/g5/07/
|-- resources/keywords.txt      Reserved words
|-- examples/example_profesor.c Professor's reading example
|-- src/main/
|   |-- lexer.py               Token and Lexer classes (core)
|   '-- main.py                Entry point (command line)
'-- tests/
    |-- conftest.py            pytest configuration
    '-- test_lexer.py          29 automated tests

```

Each token is represented by the `Token` class, which stores four fields: type, lexeme, line and column. The `Lexer` class loads the reserved words, scans the text, and keeps the valid tokens and the errors in separate lists.

5.5.2 Compiled Patterns

The patterns in Table 1 are compiled only once, when the module is loaded (Listing 1). The auxiliary pattern `_NUMERIC_CANDIDATE_RE` is deliberately more permissive than `_CONSTANT_RE`: it captures everything that looks like an attempt at a number so that it can then be validated as a whole.

Listing 1: Compiled patterns in `lexer.py`.

```
1 _NUMERIC_CANDIDATE_RE = re.compile(r"[0-9][0-9A-Za-z_]*")
2
3 _CONSTANT_RE = re.compile(r"[0-9]+(?:\.[0-9]+)?")
4 _IDENTIFIER_RE = re.compile(r"[A-Za-z_][A-Za-z0-9_]*")
5 _LITERAL_RE = re.compile(r'"[\r\n]*"')
6 _OPERATOR_RE = re.compile(r"==|!=|<=|>=|[*+<>-]")
7 _PUNCTUATION_RE = re.compile(r"([(){};],)")
```

5.5.3 Scanning Algorithm

The `tokenize()` method scans the text from left to right with an index `pos` and line and column counters. At each position it tries the following cases, in this order:

1. Newline: advances the line and resets the column to 1.
2. Space or tab: produces no token; the column advances by 1.
3. Double quote: tries to read a complete string; if it is not closed on the same line, an error is recorded.
4. Identifier: reads the longest identifier and reclassifies it as a `KEYWORD` if it belongs to the set of reserved words.
5. Numeric candidate: reads it completely and validates it against `_CONSTANT_RE`; if it does not match as a whole, it is an error.
6. Operator (two-character operators first), then punctuation.
7. Any other character: one-character error, and the scanner advances one position (panic mode).

Listing 2 shows steps 4 and 5, which contain the longest-match and reclassification rules.

Listing 2: Recognition of identifiers, keywords and constants.

```
1 # Identifiers and keywords.
2 match = _IDENTIFIER_RE.match(text, pos)
3 if match:
4     lexeme = match.group()
5     token_type = "KEYWORD" if lexeme in self.keywords else "IDENTIFIER"
6     self._add_token(token_type, lexeme, line, column)
7     pos, column = self._advance(pos, column, len(lexeme))
8     continue
9
10 # Numeric constants (and malformed numeric candidates).
11 match = _NUMERIC_CANDIDATE_RE.match(text, pos)
12 if match:
13     candidate = match.group()
14     if _CONSTANT_RE.fullmatch(candidate):
15         self._add_token("CONSTANT", candidate, line, column)
16     else:
17         self._add_error(candidate, line, column)
18     pos, column = self._advance(pos, column, len(candidate))
19     continue
```

Because `_IDENTIFIER_RE` is greedy, it reads `intx` completely before checking the set of reserved words, so it is never split into `int` and `x`. For operators, the order of the alternatives in `_OPERATOR_RE` makes `>=` be tried before `>`.

5.5.4 Lexical Error Handling

Three kinds of error are distinguished, all of them reported with their lexeme and starting line and column:

1. **Malformed number:** a candidate that starts with a digit and continues with letters, underscores or dots (123abc, 3., 1.2.3) is reported as a single error; it is never split into partially valid tokens.
2. **Unterminated string:** it is consumed up to the newline or the end of the input and reported as a single error.
3. **Illegal character:** any other character (@, &, [, accented letters) is reported and the scanner advances one character.

Errors are stored separately and are **not added** to the total number of valid tokens. The analysis always continues with the rest of the input.

5.5.5 Input and Output

The program is run from `src/main/` with `main.py`, which accepts a file or a string; with no arguments, it analyzes the professor's example:

```
python main.py                                # professor's example
python main.py --file ../../examples/example_profesor.c
python main.py --text "int a = 10;"
```

The output is a table with the type, lexeme, line and column of each token; if there are errors, a

second table lists them; and at the end the line `Total of tokens: N` is printed (plus `Total of lexical errors: M` when applicable).

6 Results

All the outputs in this section are real executions of the final version of the repository [6] (branch `main`, commit `f967b8d`). After the first example, the table headers (`TYPE LEXEME LINE COLUMN`, the dashed lines and the header of the error table) are omitted to save space; the data is unchanged.

6.1 Professor's Reading Example

Input (`examples/example_profesor.c`):

```
printf("This is an example");
int a = 10;
```

Output of `python main.py`:

TYPE	LEXEME	LINE	COLUMN

IDENTIFIER	'printf'	1	1
PUNCTUATION	'('	1	7
LITERAL	"This is an example"	1	8
PUNCTUATION	)'	1	28
PUNCTUATION	','	1	29
KEYWORD	'int'	2	1
IDENTIFIER	'a'	2	5
OPERATOR	'='	2	7
CONSTANT	'10'	2	9
PUNCTUATION	','	2	11
Total of tokens: 10			

The category sequence of the second line is `keyword identifier operator constant punctuation`, identical to the expected output shown in the statement (5 tokens). The first line produces 5 more tokens, for a total of 10. `printf` is classified as an identifier because it does not belong to the set of reserved words.

6.2 Reserved Words and Two-Character Operators

```
python main.py --text "if (x >= 10) return x; else return 0;"
```

KEYWORD	'if'	1	1
PUNCTUATION	'('	1	4
IDENTIFIER	'x'	1	5
OPERATOR	'>='	1	7
CONSTANT	'10'	1	10
PUNCTUATION	')'	1	12
KEYWORD	'return'	1	14
IDENTIFIER	'x'	1	21
PUNCTUATION	';'	1	22
KEYWORD	'else'	1	24
KEYWORD	'return'	1	29
CONSTANT	'0'	1	36
PUNCTUATION	';'	1	37
Total of tokens: 13			

The reserved words `if`, `return` and `else` are recognized as `KEYWORD`, and `>=` is a single operator.

The statement explicitly asks to present the keyword `print`:

```
python main.py --text 'print("Total"); print(a);'
```

KEYWORD	'print'	1	1
PUNCTUATION	'('	1	6
LITERAL	'"Total"'	1	7
PUNCTUATION	')'	1	14
PUNCTUATION	';'	1	15
KEYWORD	'print'	1	17
PUNCTUATION	'('	1	22
IDENTIFIER	'a'	1	23
PUNCTUATION	')'	1	24
PUNCTUATION	';'	1	25
Total of tokens: 10			

Both occurrences of `print` are reported as `KEYWORD`, while `printf` in the professor's example remains an `IDENTIFIER`.

6.3 Lexical Errors

Malformed number. `python main.py --text "int a = 123abc;"`

KEYWORD	'int'	1	1
IDENTIFIER	'a'	1	5
OPERATOR	'='	1	7
PUNCTUATION	';'	1	15
Lexical errors:			
'123abc'	1	9	
Total of tokens: 4			
Total of lexical errors: 1			

Illegal character. `python main.py --text "int a = 5 @ 3;"`

```
KEYWORD      'int'      1      1
IDENTIFIER   'a'        1      5
OPERATOR     '='        1      7
CONSTANT     '5'        1      9
CONSTANT     '3'        1     13
PUNCTUATION  ';'        1     14

Lexical errors:
'@'          1      11

Total of tokens: 6
Total of lexical errors: 1
```

Unterminated string. A file whose content is "hola (with no closing quote), analyzed with `--file:`

```
Lexical errors:
'"hola'      1      1

Total of tokens: 0
Total of lexical errors: 1
```

Non-ASCII character. `python main.py --text "int á = 10;"`

```
KEYWORD      'int'      1      1
OPERATOR     '='        1      7
CONSTANT     '10'       1      9
PUNCTUATION  ';'        1     11

Lexical errors:
'á'          1      5

Total of tokens: 4
Total of lexical errors: 1
```

In all four cases the error is reported with its position, it is not counted in the total, and the rest of the input is still analyzed.

6.4 Automated Tests

The test suite (`tests/test_lexer.py`, run with `pytest` [4]) contains 29 cases. Table 4 groups them by what they verify.

Test group	Cases
Professor's examples (5 tokens per line, 10 for both lines)	3
Priority: reserved word versus identifier (<code>if</code> , <code>if2</code> , <code>intx</code> , <code>printf</code>)	4
Operators and constants (<code>a-b</code> , <code>>=</code> , <code>-10</code> , <code>3.14</code>)	4
Malformed numbers (<code>123abc</code> , <code>3.</code> , <code>1.2.3</code> , <code>2a</code>)	4
Literals (empty string, unterminated string, string cut by a newline)	3
Undefined characters (<code>&</code> , <code>!</code> , <code>[</code> , <code>]</code> , <code>@</code>)	3
Non-ASCII characters (letter and digit)	2
Line and column (several lines, tab)	2
Empty or whitespace-only input	2
Loading reserved words from a file, and a file with a BOM	2
Total	29

Table 4: Coverage of the test suite.

```
$ pytest mx/unam/fi/compilers/g5/07/tests
collected 29 items
...
===== 29 passed in 0.08s =====
```

7 Analysis of Results

7.1 Requirements Compliance

Requirement from the statement	Evidence
Read a string or a file	Options <code>--text</code> and <code>--file</code>
Present basic tokens (keyword <code>print</code>)	Keyword <code>print</code> recognized (Section 6.2); six categories (Table 1)
Display the token sequence and the total	Section 6: <code>Total of tokens: 10</code>
Process the reading examples	Section 6 and 3 automated tests
Do not use FLEX	Custom Python scanner using the <code>re</code> module
CFG with left factoring	Section 5.4
Package <code>unam.fi.compilers.g5.07</code> and public repository	Structure <code>mx/unam/fi/compilers/g5/07</code> on GitHub

Table 5: Project requirements and evidence of compliance.

7.2 Relation between Theory and Implementation

Every decision in the program comes from a theoretical concept. The regular expressions of the specification are the patterns used by the scanner, and the DFAs in the previous section show which strings each one accepts; for example, the DFA for constants explains why `3.` is an error: it ends in state *P*, which is not accepting. The longest-match policy is what prevents `intx` or `>=` from being split. Panic mode explains why an `@` does not stop the analysis. Finally, the CFG shows how a later phase would use the token sequence produced by the lexer: the leaves of the parse tree of `int a = 10;` are exactly the five tokens the program emits for that line.

7.3 Findings during Testing

Testing found one defect, which was fixed before the final version. A second robustness measure was already part of the first version of the lexer:

- **Non-ASCII characters.** The first version chose the branch with `str.isalpha()` and `str.isdigit()`, which accept any Unicode letter or digit, while the patterns accept only ASCII. With `int á = 10;` the program terminated with an exception. It was fixed by letting the regular expression itself decide whether an identifier or a number starts at the current position; now `á` is reported as a lexical error, and two tests were added.
- **BOM at the beginning of a file.** Some Windows editors write the character `U+FEFF` at the beginning of UTF-8 files, which would otherwise be reported as an illegal character. As a precaution, files are read with the `utf-8-sig` encoding, which discards it, and a test covers this case.

7.4 Limitations

- **Typographic quotes.** The statement writes the example with curly quotes (“This is an example”). The program only recognizes straight quotes, so the example file uses `"`. With curly quotes, each one is reported as an illegal character and the words of the text are read as identifiers, so the line produces 8 tokens and 2 errors instead of 5 tokens.

- **Total of 40 tokens.** The statement mentions **Tokens:** 40 without indicating the input that produces it; with the given reading examples, the correct total is 10.
- **Language scope.** Comments, escape sequences inside strings, and operators such as `&&` or `++` are not recognized. Extending the lexer only requires adding patterns and test cases.

8 Conclusions

Team 07 built a lexical analyzer in Python that meets the requirements of the statement: it reads a string or a file, recognizes six token categories, reports the position of each token, reports lexical errors without interrupting the analysis, and displays the total number of tokens; for the professor's example it produces 5 tokens per reading line, 10 in total. The suite of 29 automated tests passes completely and includes the reading examples.

The project showed that the theory seen in class translates directly into implementation decisions. Regular expressions define the tokens; the automata, obtained through ε -closure and the subset construction, justify which strings each pattern accepts and which ones are errors; the longest match resolves conflicts between categories; and the grammar with left factoring shows how the tokens would feed the syntax analyzer, which is the next phase of the compiler.

The value of testing with edge cases also became clear: the defect that was fixed (non-ASCII characters) did not appear with the examples from the statement and was only detected by testing unusual inputs. Finally, the organization by roles, the Kanban board, and integration through pull requests made it possible to keep the specification, the code, the tests and this report consistent with each other.

References

- [1] A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, 2nd ed. Boston, MA, USA: Pearson Addison-Wesley, 2006.
- [2] J. E. Hopcroft, R. Motwani, and J. D. Ullman, *Introduction to Automata Theory, Languages, and Computation*, 3rd ed. Boston, MA, USA: Pearson Addison-Wesley, 2007.
- [3] Python Software Foundation, “re — Regular expression operations,” *Python 3 Documentation*. Accessed: Oct. 5, 2026. [Online]. Available: <https://docs.python.org/3/library/re.html>
- [4] pytest-dev, “pytest documentation.” Accessed: Oct. 5, 2026. [Online]. Available: <https://docs.pytest.org/>
- [5] Compilers, Group 5, “Lexer,” project statement, Faculty of Engineering, National Autonomous University of Mexico, Mexico City, Mexico, 2026.
- [6] Team 07, “Team-07: Lexical analyzer,” GitHub repository, 2026. [Online]. Available: <https://github.com/HansZuniga/Team-07>